

Sarathi — System Architecture Document

Version: 2.0 | **Date:** April 2026 | **Status:** Draft — Google Cloud Edition

Table of Contents

1. Architecture Overview
 2. Google Cloud Services Map
 3. System Components
 4. Data Flow Diagrams
 5. Local Model Layer
 6. Memory & RAG System
 7. Backend API Design
 8. Android App Architecture
 9. Desktop Web Architecture
 10. Cloud LLM Integration — Gemini on Vertex AI
 11. Security & Privacy Architecture
 12. Fine-Tuning Pipeline
 13. Infrastructure & Deployment
-

1. Architecture Overview

Sarathi is built on a **three-tier intelligence architecture** running entirely on Google Cloud. The core design principle is: *process sensitive data locally, enrich with personal memory, reason in the cloud.*

All cloud services are billed to Google Cloud credits — no separate billing accounts needed.



Why Google Cloud for Everything?

Concern	Google Cloud Solution
One billing account	All credits consumed from a single GCP project
LLM brain	Gemini 3.1 Pro on Vertex AI — same credits
Managed database	Cloud SQL — no PostgreSQL server to manage
Managed cache	Memorystore — fully managed Redis
Vector search	Vertex AI Matching Engine — replaces Pinecone
File storage	Google Cloud Storage — replaces S3
Push notifications	Firebase Cloud Messaging — free, deeply integrated with Android
Auth	Firebase Authentication — free, handles JWT automatically
Voice transcription	Chirp (Speech-to-Text API) — better quality than Android native
Real-time voice AI	Gemini Live API — replaces any separate TTS/STT pipeline

Concern	Google Cloud Solution
Geofencing	Google Maps Platform — most accurate on Android
CI/CD	Cloud Build — native GCP CI/CD
Secret management	Secret Manager — replaces AWS Secrets Manager
Monitoring	Cloud Logging + Cloud Monitoring — built into all GCP services

2. Google Cloud Services Map

This section maps every Sarathi component to its Google Cloud service.

Sarathi Need	Google Cloud Service	GCP Product Name
Cloud LLM brain	Gemini 3.1 Pro	Vertex AI / Gemini Enterprise Agent Platform
Real-time voice conversation	Gemini Live API	Vertex AI
Text embeddings for RAG	text-embedding-004	Vertex AI Embeddings
Vector similarity search	Matching Engine	Vertex AI
Speech-to-text (captures)	Chirp	Cloud Speech-to-Text API
Text-to-speech (Sarathi voice)	WaveNet / Neural2	Cloud Text-to-Speech API
Backend hosting	Cloud Run	Cloud Run (serverless containers)
Primary database	PostgreSQL 15	Cloud SQL
Session cache + context	Redis 7	Memorystore
File storage (images, models)	Object storage	Cloud Storage (GCS)
Push notifications (Android)	FCM	Firebase Cloud Messaging
User authentication	Firebase Auth	Firebase Authentication
Android crash reporting	Crashlytics	Firebase Crashlytics
Geofencing + Maps	Maps SDK + Geofencing API	Google Maps Platform
Places / address lookup	Places API	Google Maps Platform
Time-to-leave (routing)	Routes API	Google Maps Platform
CI/CD pipeline	Automated builds	Cloud Build

Sarathi Need	Google Cloud Service	GCP Product Name
Container registry	Docker image storage	Artifact Registry
API keys + secrets	Secrets vault	Secret Manager
Encryption keys	Key management	Cloud KMS
Logs	Centralized logs	Cloud Logging
Metrics + alerts	Monitoring dashboards	Cloud Monitoring
Scheduled jobs	Cron jobs	Cloud Scheduler + Cloud Run Jobs
Fine-tuning pipeline	Training infrastructure	Vertex AI Training (+ DGX A100 cluster)
On-device inference (Android)	TFLite + MediaPipe	MediaPipe on-device ML
On-device inference (Desktop)	Local runner	Ollama (using Gemma model files from GCS)

3. System Components

3.1 High-Level Component Map



3.2 Component Responsibilities

Component	Google Cloud Service	Responsibility
Android App	Firebase SDK + Maps SDK	Morning flow, captures, notifications, local inference
Desktop Web	React + Firebase Web SDK	Screen monitoring, focus sessions, evening debrief
Backend API	Cloud Run (FastAPI)	Gateway, business logic, orchestration
Primary DB	Cloud SQL (PostgreSQL 15)	Users, goals, tasks, schedules, habit summaries
Cache	Memorystore (Redis 7)	Session cache, conversation context, rate limiting
Vector Search	Vertex AI Matching Engine	Embedding storage + RAG similarity search
File Storage	Cloud Storage	Image captures, Gemma model files, fine-tuned checkpoints
Cloud LLM	Vertex AI Gemini 3.1 Pro	Primary reasoning and conversation brain
Voice AI	Gemini Live API	Real-time low-latency voice conversations
Speech-to-Text	Chirp (Cloud STT)	Voice capture transcription (125 languages)
Text-to-Speech	Cloud TTS	Sarathi speaking back to user
Push Notifications	Firebase Cloud Messaging	Alarm, homecoming trigger, night debrief prompt
Auth	Firebase Authentication	JWT-based login, token refresh
Geofencing	Google Maps Platform	Home detection, time-to-leave calculations
Secrets	Secret Manager	API keys, DB credentials, encryption keys
Monitoring	Cloud Logging + Monitoring	Logs, alerts, dashboards
CI/CD	Cloud Build + Artifact Registry	Automated build and deploy pipeline
Fine-Tuning	Vertex AI Training + DGX A100	Periodic personalized model updates
On-device Android	MediaPipe + TFLite	Gemma inference on phone
On-device Desktop	Ollama	Gemma inference on laptop

4. Data Flow Diagrams

4.1 Morning Wake-Up Flow

```

[Alarm Time Reached]
  |
  v
[WorkManager triggers AlarmCheck]
  |
  +-- Alarm dismissed? --YES--> [Normal day briefing starts]
  |
  NO
  v
[App opens fullscreen wake-up screen]
  |
  v
[MediaPipe Gemma (local): generate motivational prompt]
  using: goals cached in Room DB (no network needed)
  |
  v
[User confirms awake -- taps interaction]
  |
  v
[FastAPI on Cloud Run: GET /lifecycle/morning]
  |
  v
[Memory Service: Vertex AI Matching Engine query]
  fetch: goals, gym plan, schedule, pending captures
  |
  v
[Vertex AI Gemini 3.1 Pro: generate morning briefing]
  context: goals + schedule + retrieved memory
  |
  v
[Google Maps Routes API: calculate time-to-leave]
  home --> gym --> college route + current traffic
  |
  v
[Android renders structured briefing]
  - Top goals for today
  - Workout plan
  - Breakfast suggestion
  - Live countdown: time to leave (updated via Maps API)

```

4.2 Screen Monitoring Flow (Laptop)

```
[User starts focus session on laptop]
  |
  v
[Desktop web app: browser Screen Capture API]
  |
  v  every 30 seconds
[Frame sent to LOCAL Ollama (Gemma 7B)]
  -- raw frames NEVER leave the device --
  |
  v
[Local Gemma classifies activity]
  --> PRODUCTIVE: coding, studying, reading, writing
  --> DISTRACTION: social media, games, entertainment
  |
  +-- PRODUCTIVE --> [Log focus time, no interruption]
  |
  DISTRACTION for > 5 consecutive minutes
  v
[Nudge overlay appears]
  message generated by local Gemma referencing user's current goal
  |
  v
[Local model extracts habit summary text]
  e.g. "User distracted by YouTube ~3x during session"
  raw frames discarded immediately
  |
  v
[FastAPI Cloud Run: POST /habits/summary]
  |
  v
[Cloud SQL: habit_summaries table updated]
[Vertex AI Embedding: summary embedded + stored in Matching Engine]
```

4.3 RAG Memory Retrieval Flow


```
[Chat request arrives at FastAPI (Cloud Run)]
  |
  v
[Memory Service: embed current message]
  Vertex AI text-embedding-004 API
  |
  v
[Vertex AI Matching Engine: top-k query]
  filter: user_id = current user
  returns: relevant habits, captures, goals, past summaries
  |
  v
[Memorystore Redis: fetch last 20 conversation turns]
  |
  v
[Assemble enriched Gemini prompt]
  System: Sarathi persona + lifecycle phase
  Memory: Matching Engine results
  History: Redis conversation cache
  User:   current message
  |
  v
[Vertex AI Gemini 3.1 Pro: generate response]
  |
  v
[Response streamed back via WebSocket]
  |
  v
[Turn stored in Memorystore Redis + Cloud SQL]
[Turn embedded + stored in Vertex AI Matching Engine]
```

4.4 Evening Homecoming Flow

```

[Google Maps Geofence API: HOME radius entered]
  |
  v
[Android: Firebase Cloud Messaging trigger]
  push notification: "Hey, you're back. Let's talk."
  |
  v
[FastAPI Cloud Run: POST /lifecycle/homecoming]
  |
  v
[Cloud SQL: fetch all captures from today]
[Vertex AI Matching Engine: fetch day's habit summaries]
  |
  v
[Vertex AI Gemini 3.1 Pro: generate homecoming opener]
  references: specific captures, morning goals, focus session data
  |
  v
[Conversational debrief loop]
  Voice: Chirp STT --> FastAPI --> Gemini --> Cloud TTS --> user
  Text: type --> FastAPI --> Gemini --> text response
  |
  v
[After debrief: generate evening work plan]
  - pending tasks from captures
  - goal-aligned suggestions
  |
  v
[User transitions to laptop work session]

```

4.5 Voice Capture Flow (Quick Capture with Chirp)

```

[User taps mic button in Android app]
  |
  v
[Android: SpeechRecognizer (on-device, fast)]
  -- for short captures during class --
  |
  v
[Audio streamed to Cloud Speech-to-Text (Chirp)]
  -- for longer captures, better accuracy --
  |
  v
[Transcript returned]
  |
  v
[FastAPI Cloud Run: POST /captures]
  {content: transcript, input_type: "voice"}
  |
  v
[Vertex AI Gemini Flash: quick auto-categorize]
  e.g. "task", "reminder", "observation", "question"
  |
  v
[Cloud SQL: captures table saved]
[Vertex AI Matching Engine: capture embedded + indexed]

```

5. Local Model Layer

5.1 Model Variants

Variant	Device	Base Model	Approx Size	Runtime	Source
Sarathi-Mini	Android	Gemma 2B INT4 quantized	~1.5 GB	MediaPipe / TFLite	Downloaded from GCS
Sarathi-Max	Laptop	Gemma 7B Q4_K_M quantized	~4.5 GB	Ollama	Downloaded from GCS

Model files stored in **Google Cloud Storage (GCS)**. Android `ModelUpdateWorker` pulls new versions from GCS automatically when on Wi-Fi and charging.

5.2 What the Local Model Does

- **Screen Classification** — productive vs distraction (laptop only, via Ollama)
- **Habit Extraction** — converts raw activity into summarized habit text
- **Privacy Gating** — decides what is safe to send to Vertex AI
- **Wake-up Motivation** — generates goal-tied prompts offline (Room DB as context)
- **Quick Capture Pre-processing** — lightweight tagging before Chirp or Gemini call

5.3 Local Model API Contract

```
Android (MediaPipe):
  MediaPipe LlmInference.generateResponse(prompt) -> String

Desktop (Ollama):
POST http://localhost:11434/api/generate
{
  "model": "gemma:7b",
  "prompt": "<task_specific_prompt>",
  "stream": false
}
Response: { "response": "<output>", "done": true }
```

5.4 Battery & Performance Strategy

Mode	Behavior	User Control
Always On	Local model runs continuously	Toggle in Settings
Smart (default)	Runs only on charge or battery > 50%	Default
Manual Only	Runs only when app is open	Toggle in Settings

6. Memory & RAG System

6.1 Cloud SQL Schema (PostgreSQL 15 on Google Cloud SQL)

```
-- Users
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  firebase_uid TEXT UNIQUE NOT NULL,    -- Firebase Auth UID
  name TEXT NOT NULL,
  email TEXT UNIQUE NOT NULL,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Goals
CREATE TABLE goals (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,
  title TEXT NOT NULL,
  description TEXT,
  deadline DATE,
  priority INT DEFAULT 1,
  is_active BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Daily Schedule
CREATE TABLE schedules (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,
  day_of_week INT,          -- 0=Mon ... 6=Sun
  wake_time TIME,
  gym_time TIME,
  college_leave_time TIME,
  sleep_target_time TIME,
  home_lat FLOAT,
  home_lng FLOAT
);

-- Task Captures
CREATE TABLE captures (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,
  content TEXT,
  input_type TEXT CHECK (input_type IN ('voice', 'text', 'image')),
```

```
gcs_image_url TEXT,          -- GCS signed URL if image
category TEXT,              -- auto-classified by Gemini Flash
is_resolved BOOLEAN DEFAULT FALSE,
captured_at TIMESTAMPTZ DEFAULT NOW()
);

-- Habit Summaries (from local model)
CREATE TABLE habit_summaries (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,
  summary TEXT NOT NULL,
  habit_type TEXT,
  recorded_at TIMESTAMPTZ DEFAULT NOW()
);

-- Conversation History
CREATE TABLE conversation_turns (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,
  lifecycle_phase TEXT,
  role TEXT CHECK (role IN ('user', 'assistant')),
  content TEXT NOT NULL,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Alarms
CREATE TABLE alarms (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,
  alarm_time TIME NOT NULL,
  is_active BOOLEAN DEFAULT TRUE,
  set_by TEXT DEFAULT 'sarathi',
  set_at TIMESTAMPTZ DEFAULT NOW()
);
```

6.2 Vector Memory — Vertex AI Matching Engine

Replaces Pinecone entirely. Billed to Google Cloud credits.

```

# Embedding a memory item using Vertex AI
from vertexai.language_models import TextEmbeddingModel

model = TextEmbeddingModel.from_pretrained("text-embedding-004")

def embed(text: str) -> list[float]:
    embeddings = model.get_embeddings([text])
    return embeddings[0].values # 768-dim vector

# Storing in Matching Engine
def store_memory(user_id: str, content: str, memory_type: str):
    vector = embed(content)
    index.upsert([
        {
            "id": f"{user_id}_{uuid4()}",
            "embedding": vector,
            "restricts": [{"namespace": "user_id", "allow": [user_id]}],
            "crowding_tag": memory_type
        }
    ])

# Retrieving relevant memory
def retrieve_memory(user_id: str, context: str, top_k: int = 8) -> list[str]:
    query_vector = embed(context)
    results = index.find_neighbors(
        query=query_vector,
        num_neighbors=top_k,
        restricts=[{"namespace": "user_id", "allow": [user_id]}]
    )
    return [r.metadata["content"] for r in results]

```

6.3 Memorystore (Redis) Cache Schema

sarathi:conv:{user_id}	--> JSON list of last 20 turns	(TTL: 2 hours)
sarathi:session:{user_id}	--> Firebase JWT payload	(TTL: 1 hour)
sarathi:ratelimit:{user_id}	--> request count per minute	(TTL: 60s)
sarathi:phase:{user_id}	--> morning day evening night	
sarathi:briefing:{user_id}	--> today's briefing cache	(TTL: 12 hours)

6.4 Cloud Storage (GCS) Bucket Structure

```

gs://sarathi-app/
+-- user-images/
|   +-- {user_id}/
|       +-- {capture_id}.jpg      (image captures)
+-- model-checkpoints/
|   +-- sarathi-mini/
|       |   +-- v1.0.tflite
|       |   +-- v1.1.tflite      (updated fine-tuned versions)
|   +-- sarathi-max/
|       +-- v1.0.gguf
|       +-- v1.1.gguf
+-- training-data/
    +-- {user_id}/
        +-- dataset_{date}.jsonl (fine-tuning datasets, encrypted)

```

7. Backend API Design

7.1 Hosting

FastAPI deployed on **Google Cloud Run** — serverless, auto-scales, pays per request. No idle cost.

```
Base URL: https://api.sarathi.app/v1
Hosting:  Cloud Run (us-central1 region)
Auth:    Firebase Authentication (JWT verification via Firebase Admin SDK)
```

7.2 Auth — Firebase Admin SDK

```
import firebase_admin
from firebase_admin import auth

def verify_token(token: str) -> dict:
    decoded = auth.verify_id_token(token)
    return decoded    # contains uid, email, etc.

# FastAPI dependency
async def get_current_user(authorization: str = Header(...)):
    token = authorization.replace("Bearer ", "")
    return verify_token(token)
```

No custom JWT logic needed — Firebase handles it.

7.3 Core API Endpoints

AUTH (handled by Firebase client SDK – no backend endpoints needed)

LIFECYCLE

GET	/lifecycle/phase	Get current phase
POST	/lifecycle/morning	Trigger morning briefing
POST	/lifecycle/homecoming	Trigger homecoming debrief
POST	/lifecycle/night	Trigger night debrief

CONVERSATION

POST	/chat/message	Send message, get Gemini response (streamed)
GET	/chat/history	Fetch conversation history from Cloud SQL
DELETE	/chat/history	Clear conversation history

CAPTURES

POST	/captures	Create capture (text/voice/image)
POST	/captures/image	Upload image to GCS, create capture
GET	/captures	List captures (filter by date/category)
PATCH	/captures/{id}	Mark capture resolved
DELETE	/captures/{id}	Delete capture

GOALS & SCHEDULE

GET	/goals	List active goals
POST	/goals	Create goal
PATCH	/goals/{id}	Update goal
DELETE	/goals/{id}	Delete goal
GET	/schedule	Get daily schedule + home location
PUT	/schedule	Update schedule

ALARM

GET	/alarm	Get current alarm
POST	/alarm	Set alarm (called after night debrief)
DELETE	/alarm	Cancel alarm

HABITS

POST	/habits/summary	Submit habit summary from local model
GET	/habits/summary	Get recent habit summaries

MAPS

GET	/maps/time-to-leave	Get commute time to destination (Routes API)
POST	/maps/home	Save home location (lat/lng)

7.4 Chat Message — Image Upload Flow

```
# Image capture upload to GCS
async def upload_image_to_gcs(file: UploadFile, user_id: str) -> str:
    client = storage.Client()
    bucket = client.bucket("sarathi-app")
    blob = bucket.blob(f"user-images/{user_id}/{uuid4()}.jpg")
    blob.upload_from_file(file.file, content_type="image/jpeg")
    # Return signed URL valid for 7 days
    return blob.generate_signed_url(expiration=timedelta(days=7))
```

7.5 Chat Streaming with Gemini


```
import vertexai
from vertexai.generative_models import GenerativeModel

vertexai.init(project="sarathi-gcp", location="us-central1")

async def chat_stream(user_id: str, message: str, phase: str):
    memory = await retrieve_memory(user_id, message)
    history = await get_conversation_history(user_id) # from Memorystore

    system_prompt = f"""
You are Sarathi -- a personal lifecycle AI assistant named after the charioteer
who guided Arjuna in the Mahabharata. You guide this user through their day.
You are firm, honest, and personal. No hollow motivation.

Phase: {phase} | Date: {today}
User memory:
{chr(10).join(f'- {m}' for m in memory)}
"""
    model = GenerativeModel(
        model_name="gemini-2.5-pro-preview-0325",
        system_instruction=system_prompt
    )
    chat = model.start_chat(history=history)
    response = chat.send_message(message, stream=True)

    for chunk in response:
        yield chunk.text # WebSocket stream to client
```

8. Android App Architecture

8.1 Module Structure

```
app/
+-- di/                Hilt modules
+-- data/
| +-- local/
| | +-- dao/           Room DAOs (local cache)
| | +-- entity/        Room entities
| | +-- SarathiDatabase.kt
| +-- remote/
| | +-- api/           Retrofit interfaces (FastAPI)
| | +-- dto/           Data transfer objects
| +-- repository/      Repository implementations
+-- domain/
| +-- model/           Domain models
| +-- repository/      Repository interfaces
| +-- usecase/         Use cases
+-- ui/
| +-- morning/         Morning briefing screens
| +-- capture/         Quick capture (voice/text/image)
| +-- chat/            Conversation screen
| +-- evening/         Evening debrief
| +-- night/           Night debrief + alarm
| +-- settings/        User settings + home location
+-- service/
| +-- AlarmService.kt  Alarm monitoring
| +-- LocationService.kt Google Maps Geofencing
| +-- LocalModelService.kt MediaPipe Gemma inference
| +-- ChirpService.kt  Cloud Speech-to-Text
+-- worker/
| +-- AlarmCheckWorker.kt
| +-- HabitSyncWorker.kt
| +-- ModelUpdateWorker.kt Pulls new Gemma versions from GCS
```

8.2 Key Android Libraries & Services

Component	Library / Service	Purpose
UI	Jetpack Compose	All screens
DI	Hilt	Dependency injection
Auth	Firebase Authentication SDK	Login, JWT
Push Notifications	Firebase Cloud Messaging	Alarm, homecoming, night debrief triggers
Crash Reporting	Firebase Crashlytics	Production crash tracking
Background Jobs	WorkManager	Alarm checks, habit sync, model updates
Alarm	AlarmManager (exact)	Precise alarm trigger
Location	Google Maps Fused Location + Geofencing API	Home detection
Maps	Maps SDK for Android + Routes API	Time-to-leave countdown

Component	Library / Service	Purpose
Voice Capture	Android SpeechRecognizer (short) + Cloud Chirp (long)	Transcription
Image Capture	CameraX	Photo capture
Local Gemma	MediaPipe LLM Inference	On-device model
Model Download	WorkManager + GCS download	Pulls updated fine-tuned models
Local DB	Room	Offline cache of goals, schedule, captures
Networking	Retrofit + OkHttp + WebSocket	FastAPI communication

8.3 Firebase Integration in Android

```
// Firebase Auth – login
FirebaseAuth.getInstance()
    .signInWithEmailAndPassword(email, password)
    .addOnSuccessListener { result ->
        result.user?.getIdToken(true)?.addOnSuccessListener { tokenResult ->
            // Store token, use as Bearer in all API calls
            tokenStore.save(tokenResult.token)
        }
    }
}

// Firebase Cloud Messaging – receive homecoming trigger
class SarathiMessagingService : FirebaseMessagingService() {
    override fun onMessageReceived(message: RemoteMessage) {
        when (message.data["type"]) {
            "homecoming" -> launchEveningDebrief()
            "night_debrief"-> launchNightDebrief()
            "wake_up"     -> launchWakeUpScreen()
        }
    }
}
```

8.4 Google Maps Geofencing Setup

```
val geofence = Geofence.Builder()
    .setRequestId("HOME")
    .setCircularRegion(homeLat, homeLng, 100f) // 100 meter radius
    .setExpirationDuration(Geofence.NEVER_EXPIRE)
    .setTransitionTypes(Geofence.GEOFENCE_TRANSITION_ENTER)
    .build()

val request = GeofencingRequest.Builder()
    .setInitialTrigger(GeofencingRequest.INITIAL_TRIGGER_ENTER)
    .addGeofence(geofence)
    .build()

geofencingClient.addGeofences(request, geofencePendingIntent)
```

8.5 Lifecycle Phase State Machine

```

+-----+
|  MORNING  | <-- Alarm time + WorkManager
+-----+
      | User confirms awake
+-----V-----+
|    DAY    | <-- User leaves home geofence
+-----+
      | User enters home geofence (Maps API)
+-----V-----+
|  EVENING  | <-- FCM push notification
+-----+
      | Work session complete / time pattern
+-----V-----+
|   NIGHT   | <-- FCM push / schedule time
+-----+
      | Alarm set via AlarmManager
+-----> MORNING (next day)

```

9. Desktop Web Architecture

9.1 Tech Stack

```

Frontend: React 18 + TypeScript
Styling:  Tailwind CSS
State:    Zustand
API:      Axios + WebSocket (socket.io-client)
Auth:     Firebase Authentication Web SDK
Local AI: Ollama REST API (localhost:11434)
Build:    Vite
Hosting:   Firebase Hosting (static) or Cloud Run

```

9.2 Firebase Auth on Web

```

import { getAuth, signInWithEmailAndPassword } from 'firebase/auth';

const auth = getAuth();

async function login(email: string, password: string) {
  const result = await signInWithEmailAndPassword(auth, email, password);
  const token = await result.user.getIdToken();
  // Use token as Bearer for all FastAPI calls
  apiClient.defaults.headers.common['Authorization'] = `Bearer ${token}`;
}

```

9.3 Screen Monitoring — Local Ollama

```

async function startScreenMonitoring() {
  const stream = await navigator.mediaDevices.getDisplayMedia({
    video: { frameRate: 1 }
  });
  const video = document.createElement('video');
  video.srcObject = stream;
  await video.play();

  setInterval(async () => {
    const canvas = document.createElement('canvas');
    canvas.width = video.videoWidth;
    canvas.height = video.videoHeight;
    canvas.getContext('2d')!.drawImage(video, 0, 0);
    const base64 = canvas.toDataURL('image/jpeg', 0.4);

    // Sent to LOCAL Ollama only -- never to Vertex AI or cloud
    const res = await fetch('http://localhost:11434/api/generate', {
      method: 'POST',
      body: JSON.stringify({
        model: 'gemma:7b',
        prompt: `You are a focus monitor. Look at this screen and classify it as exactly one word:
PRODUCTIVE (if coding, studying, reading, writing, working)
DISTRACTION (if social media, YouTube, games, entertainment, messaging)
Screen: ${base64}
Reply with one word only.`
      })
    });
    const data = await res.json();
    const result = data.response.trim().toUpperCase();

    if (result === 'DISTRACTION') {
      distractionCount++;
      if (distractionCount >= 10) { // 10 x 30s = 5 minutes
        showNudge();
        await logDistraction();
        distractionCount = 0;
      }
    } else {
      distractionCount = 0;
      logFocusTime();
    }
  }, 30_000);
}

async function logDistraction() {
  // Only the habit summary goes to the backend -- not the frame
  await api.post('/habits/summary', {
    summary: `Distraction detected during focus session at ${new Date().toISOString()}`,
    habit_type: 'distraction'
  });
}

```

9.4 Component Structure

```

src/
+-- components/
|   +-- Chat/           Streaming conversation UI
|   +-- Morning/        Morning briefing view
|   +-- FocusSession/   Work session + screen monitor
|   +-- Evening/        Homecoming debrief
|   +-- Night/          Night debrief + alarm setup
|   +-- Settings/       Goals, schedule, preferences
+-- store/
|   +-- chatStore.ts
|   +-- lifecycleStore.ts
|   +-- userStore.ts
+-- services/
|   +-- api.ts           FastAPI (Cloud Run) client
|   +-- socket.ts        WebSocket client
|   +-- localModel.ts    Ollama client
|   +-- firebase.ts      Firebase Auth + FCM web
+-- hooks/
    +-- useLifecycle.ts
    +-- useScreenMonitor.ts
    +-- useCapture.ts

```

10. Cloud LLM Integration — Gemini on Vertex AI

10.1 Model Selection

Use Case	Model	Why
Main conversations (morning, evening, night debrief)	gemini-2.5-pro	Best reasoning, personalized responses
Quick capture auto-categorization	gemini-2.5-flash-lite	Low latency, cheap, high volume
Real-time voice conversations	Gemini Live API (2.5 Flash Native Audio)	Sub-second latency, natural voice
Text embeddings for RAG	text-embedding-004	Best quality, 768 dims
Screen nudge message	Local Gemma (Ollama)	Private, no cloud needed

10.2 Gemini Live API — Voice Conversation Mode

For the morning briefing and evening debrief, Sarathi can optionally use **Gemini Live API** for a fully voice-native experience — no separate STT + TTS pipeline needed.

```
# Gemini Live API – real-time voice session
from google import genai

client = genai.Client(vertexai=True, project="sarathi-gcp", location="us-central1")

async def start_voice_session(user_id: str, phase: str):
    memory = await retrieve_memory(user_id, f"user lifecycle phase {phase}")

    config = {
        "response_modalities": ["AUDIO"],
        "system_instruction": f"""
You are Sarathi, a personal AI charioteer. Phase: {phase}.
User context: {memory}
Be direct. Be personal. No hollow motivation.
        """
    }

    async with client.aio.live.connect(
        model="gemini-2.5-flash-native-audio",
        config=config
    ) as session:
        # Bidirectional audio stream
        async for audio_chunk in user_audio_stream():
            await session.send(audio=audio_chunk)
            async for response in session.receive():
                yield response.audio # stream back to user
```

10.3 Standard Text Prompt Architecture

```
import vertexai
from vertexai.generative_models import GenerativeModel, Content, Part

vertexai.init(project="sarathi-gcp", location="us-central1")

def build_history(turns: list[dict]) -> list[Content]:
    return [
        Content(role=t["role"], parts=[Part.from_text(t["content"])])
        for t in turns[-10:]
    ]

async def chat_stream(user_id: str, message: str, phase: str):
    memory = await retrieve_memory(user_id, message)
    history = await get_redis_history(user_id)

    model = GenerativeModel(
        model_name="gemini-2.5-pro-preview-0325",
        system_instruction=f"""
You are Sarathi -- named after the charioteer who guided Arjuna.
You guide this user toward their goals every single day.
Be honest. Be firm. Be personal. Reference their actual goals and history.

Lifecycle phase: {phase}
What you know about this user:
{chr(10).join(f'- {m}' for m in memory)}
"""
    )

    chat = model.start_chat(history=build_history(history))
    response = chat.send_message(message, stream=True)

    for chunk in response:
        yield chunk.text
```

11. Security & Privacy Architecture

11.1 Data Classification

Data Type	Stored Where	Encrypted	Sent to Cloud?
Raw screen frames	Nowhere (discarded after local analysis)	N/A	Never
Voice audio	Nowhere (transcribed, discarded)	N/A	Never
Habit summaries (text only)	Cloud SQL	Yes	Yes
Task captures	Cloud SQL + Vertex AI Matching Engine	Yes	Yes
Goals	Cloud SQL	Yes	Yes

Data Type	Stored Where	Encrypted	Sent to Cloud?
Conversation text	Memorystore + Cloud SQL	Yes	Yes
Image captures	Google Cloud Storage	Yes (GCS server-side)	URL only
Model checkpoints	Google Cloud Storage	Yes (GCS server-side)	Download only

11.2 Google Cloud Security Services Used

Security Need	Google Cloud Service
API key and credential storage	Secret Manager
Per-user encryption keys	Cloud KMS
User authentication	Firebase Authentication
Backend auth verification	Firebase Admin SDK
Data in transit	TLS 1.3 (automatic on Cloud Run and GCP APIs)
Data at rest	AES-256 (default on Cloud SQL, GCS, Memorystore)
Access control	Cloud IAM (least privilege per service account)
API access control	Cloud Endpoints + API Gateway

11.3 Secret Manager — Key Variables

```

sarathi/vertex-ai-key      --> GCP service account key for Vertex AI
sarathi/db-password       --> Cloud SQL password
sarathi/firebase-config   --> Firebase Admin SDK credentials
sarathi/maps-api-key      --> Google Maps Platform API key
sarathi/gcs-bucket-name   --> GCS bucket name
sarathi/jwt-secret        --> Additional signing secret
sarathi/kms-key-id        --> Cloud KMS key for user data encryption

```

11.4 Firebase Auth Flow

```

Android / Web Client
  |
  | signInWithEmailAndPassword()
  v
Firebase Authentication
  |
  | ID Token (JWT, 1 hour TTL)
  v
FastAPI on Cloud Run
  |
  | firebase_admin.auth.verify_id_token(token)
  v
Verified user UID -- all DB queries filtered by this UID

```

12. Fine-Tuning Pipeline

12.1 Overview

```

User data accumulates in Cloud SQL + Vertex AI Matching Engine
  |
  v Weekly trigger via Cloud Scheduler
Cloud Run Job: export + anonymize + format as JSONL dataset
  |
  v
Upload dataset to GCS: gs://sarathi-app/training-data/{user_id}/
  |
  v
Vertex AI Training Job (OR DGX A100 cluster for large batches)
  - LoRA fine-tuning on Gemma 2B / 7B
  - HuggingFace Transformers + PEFT
  |
  v
Evaluation: perplexity + response quality checks
  |
  v
Export: GGUF (Ollama desktop) + TFLite (Android MediaPipe)
  |
  v
Upload to GCS: gs://sarathi-app/model-checkpoints/
  |
  v
Android ModelUpdateWorker: detects new version, downloads on Wi-Fi + charging
Desktop: Ollama auto-pulls new model file from GCS URL

```

12.2 Vertex AI Training Job Config

```
from google.cloud import aiplatform

aiplatform.init(project="sarathi-gcp", location="us-central1")

job = aiplatform.CustomTrainingJob(
    display_name=f"sarathi-finetune-{user_id}-{date}",
    script_path="train/finetune_gemma.py",
    container_uri="us-docker.pkg.dev/vertex-ai/training/pytorch-gpu.2-0:latest",
    requirements=["transformers", "peft", "datasets"],
)

job.run(
    dataset=aiplatform.TextDataset(gcs_source=f"gs://sarathi-app/training-data/{user_id}/"),
    replica_count=1,
    machine_type="a2-highgpu-1g",          # A100 40GB for Vertex AI
    accelerator_type="NVIDIA_TESLA_A100",
    accelerator_count=1,
)
```

For larger batches, use the college DGX A100 cluster instead and push outputs to GCS.

12.3 Training Approach

Parameter	Value
Method	LoRA (Low-Rank Adaptation) via PEFT
Base model (mobile)	Gemma 2B
Base model (desktop)	Gemma 7B
Training data	Conversation history, habit summaries, goal interactions
Frequency	Weekly (Phase 3+)
Output format (desktop)	GGUF — Ollama compatible
Output format (mobile)	TFLite — MediaPipe compatible
Storage	Google Cloud Storage
Trigger	Cloud Scheduler --> Cloud Run Job

13. Infrastructure & Deployment

13.1 Google Cloud Project Structure

```

GCP Project: sarathi-prod
+-- Cloud Run:          sarathi-api (FastAPI backend)
+-- Cloud SQL:          sarathi-db (PostgreSQL 15)
+-- Memorystore:        sarathi-cache (Redis 7)
+-- Cloud Storage:      sarathi-app (images, models, training data)
+-- Vertex AI:          Gemini 3.1 Pro + Matching Engine + Embeddings
+-- Firebase:           Authentication + FCM + Crashlytics
+-- Secret Manager:     All credentials
+-- Cloud Build:        CI/CD pipeline
+-- Artifact Registry:  Docker images
+-- Cloud Scheduler:    Fine-tuning trigger (weekly)
+-- Cloud Logging:       All service logs
+-- Cloud Monitoring:    Dashboards + alerts
+-- Google Maps:         Geofencing + Routes + Places

```

13.2 Cloud Run Deployment (FastAPI)

```

# cloudbuild.yaml -- Cloud Build CI/CD
steps:
  - name: 'python:3.11'
    entrypoint: pip
    args: ['install', '-r', 'requirements.txt']

  - name: 'python:3.11'
    entrypoint: pytest
    args: ['tests/']

  - name: 'gcr.io/cloud-builders/docker'
    args: ['build', '-t', 'us-central1-docker.pkg.dev/sarathi-prod/sarathi/api:$COMMIT_SHA', '.']

  - name: 'gcr.io/cloud-builders/docker'
    args: ['push', 'us-central1-docker.pkg.dev/sarathi-prod/sarathi/api:$COMMIT_SHA']

  - name: 'gcr.io/google.com/cloudsdktool/cloud-sdk'
    args:
      - gcloud
      - run
      - deploy
      - sarathi-api
      - --image=us-central1-docker.pkg.dev/sarathi-prod/sarathi/api:$COMMIT_SHA
      - --region=us-central1
      - --allow-unauthenticated
      - --set-secrets=DB_PASSWORD=sarathi/db-password:latest
      - --set-secrets=VERTEX_KEY=sarathi/vertex-ai-key:latest
      - --set-secrets=MAPS_KEY=sarathi/maps-api-key:latest

```

13.3 Environment Variables (from Secret Manager)

```

# Injected automatically by Cloud Run from Secret Manager
GCP_PROJECT_ID=sarathi-prod
VERTEX_AI_LOCATION=us-central1
DB_CONNECTION_NAME=sarathi-prod:us-central1:sarathi-db
REDIS_HOST=<memorystore-ip>
GCS_BUCKET=sarathi-app
FIREBASE_PROJECT_ID=sarathi-prod
MAPS_API_KEY=<from Secret Manager>

```

13.4 Estimated Monthly Google Cloud Credit Usage (MVP Phase)

Service	Estimated Monthly Cost
Cloud Run (FastAPI backend)	~\$5–15 (low traffic MVP)
Cloud SQL (PostgreSQL, db-f1-micro)	~\$10
Memorystore (Redis, 1GB)	~\$25
Vertex AI Gemini 3.1 Pro (conversations)	~\$20–50 (depends on usage)
Vertex AI Matching Engine	~\$10
Cloud Storage	~\$1–3
Firebase (Auth + FCM)	Free tier
Google Maps Platform	Free tier (28,000 map loads/month)
Cloud Build	Free tier (120 min/day)
Secret Manager	~\$1
Total MVP estimate	~\$72–115/month

Credits will cover this comfortably during development and testing.

Sarathi Architecture Document v2.0 | April 2026 | Google Cloud Edition | Confidential